How Does a Coding Agent Work?



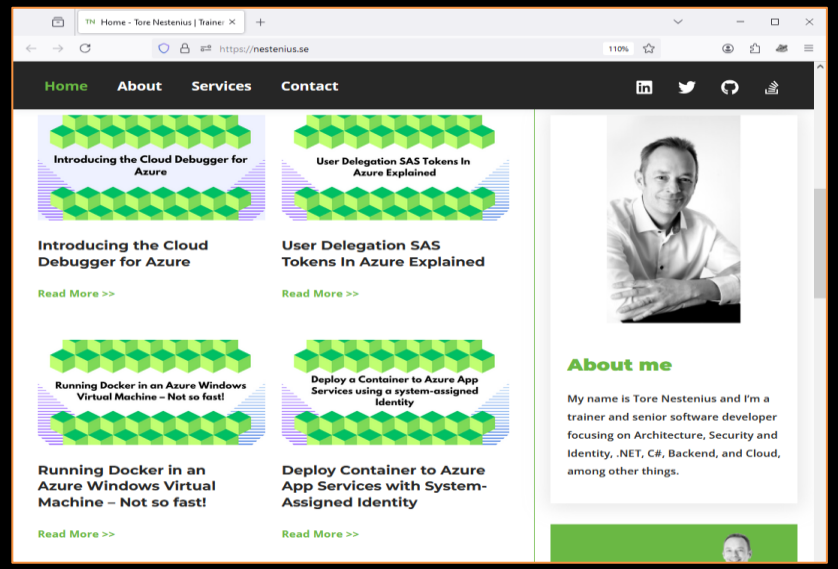
nestenius.se



About Tore Nestenius



Work: tn-data.se



Blog: nestenius.se

.NET Skåne

meetup.com/net-skane

Have a question about this session or beyond?

✉ **tore@tn-data.se**

First, One Important Thing

Do Ask Questions!

How many of you…

Claude Code,
Copilot, Cursor…

Have used a **Coding Agent**?

Could **explain** how it actually works?

Why Did I Write This Talk?

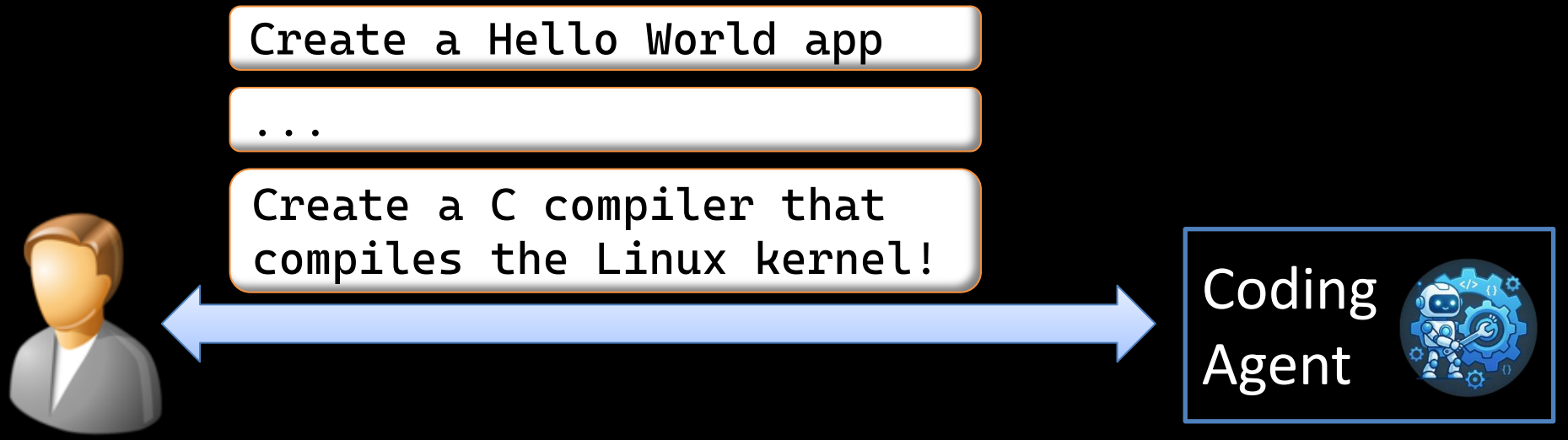
I’m building an Agentic AI Development workshop



But how do you actually teach agentic dev?

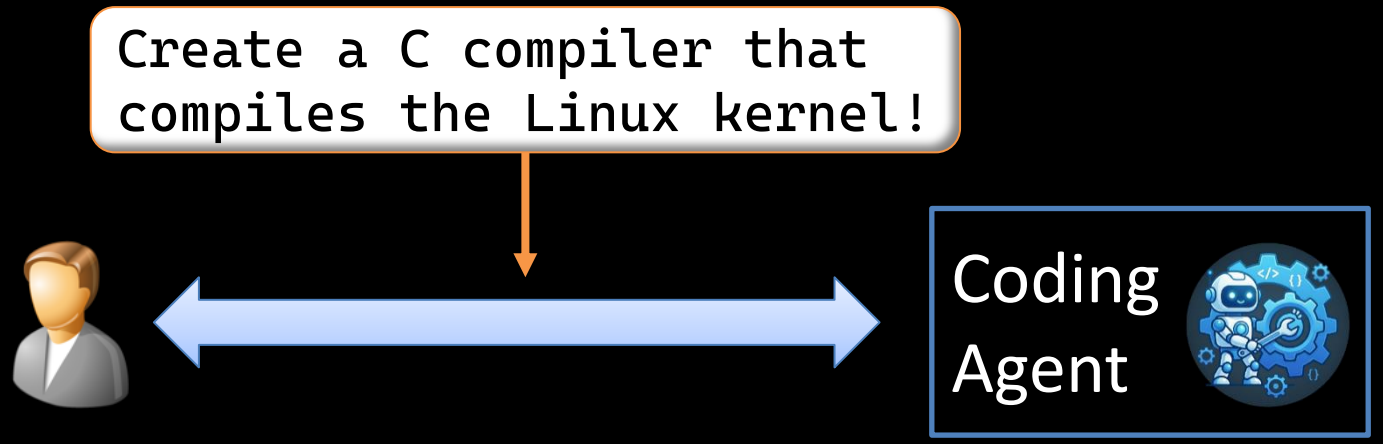
https://nestenius.se

This is what most people are learning today



Does this actually teach you agentic coding?

What you actually get



- Expensive
- Unreliable results
- Lack of understanding

What do we need instead?

What we actually need is **mechanical sympathy**

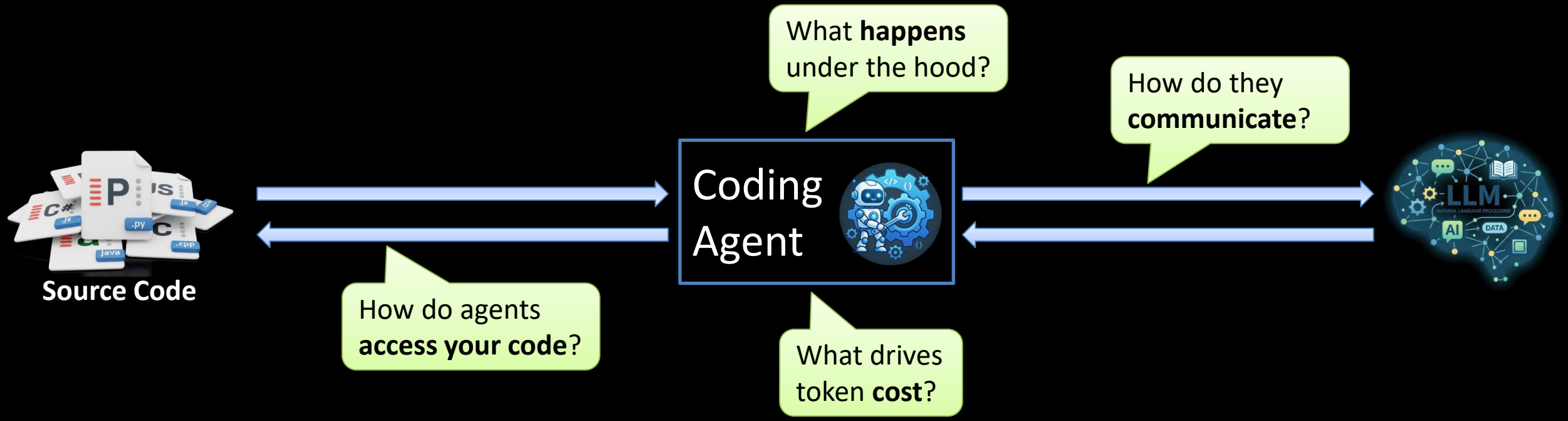




You don't drive a race car by ignoring the engine

Work with the system, not against it!

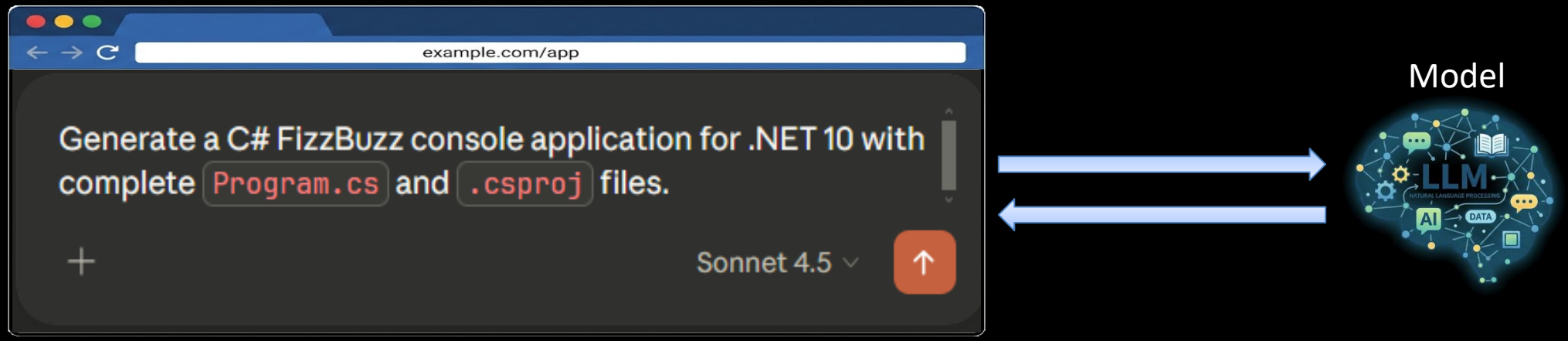
Mechanical Sympathy = understanding the system



Let’s look at how this works

From Prompting to Agentic Development

Most of us started by using LLMs in the browser



Why doesn’t this scale to real software work?

The developer becomes the message transport

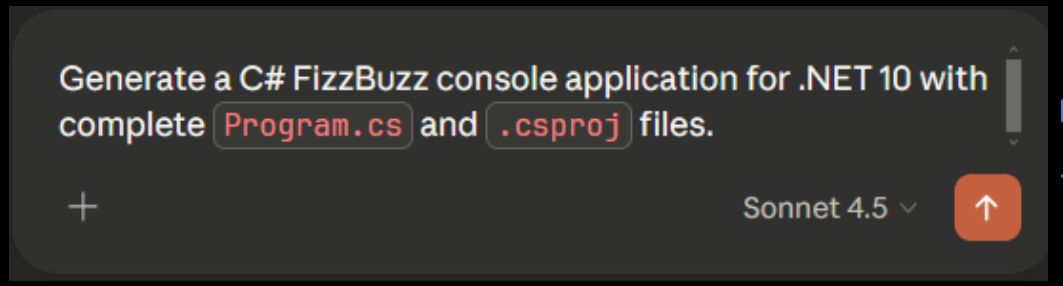
Source Code



Copy / Paste



Web Browser



Model

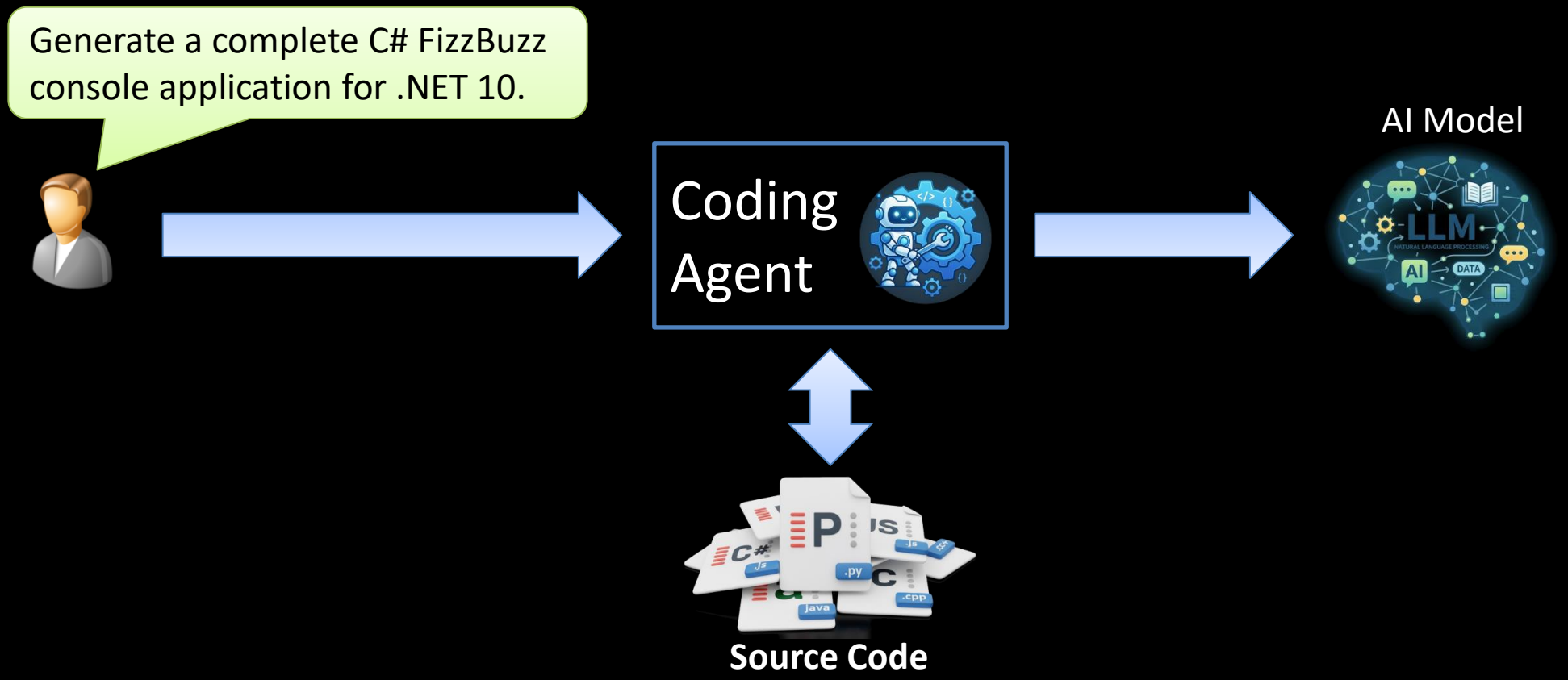




Copy -> Paste -> Error -> Copy -> Paste -> Repeat...

Can we remove the human from the feedback loop?

Yes, you can remove the human from the loop:



What's happening under the hood?

First, let’s look at how the agent talks to the model



How can we observe this traffic?

Demo #1



HTTP Inspector

Table view of all proxied requests with headers, bodies, SSE events, and timing details.

Conversation View

Chat-style display of Claude conversations showing messages, tool use, and responses.

How does the agent access the source code?



Source Code

Large Language Models (LLMs) operate only on text



Source Code

Coding Agent



`what is the problem with the code in my **Program.cs** file?`



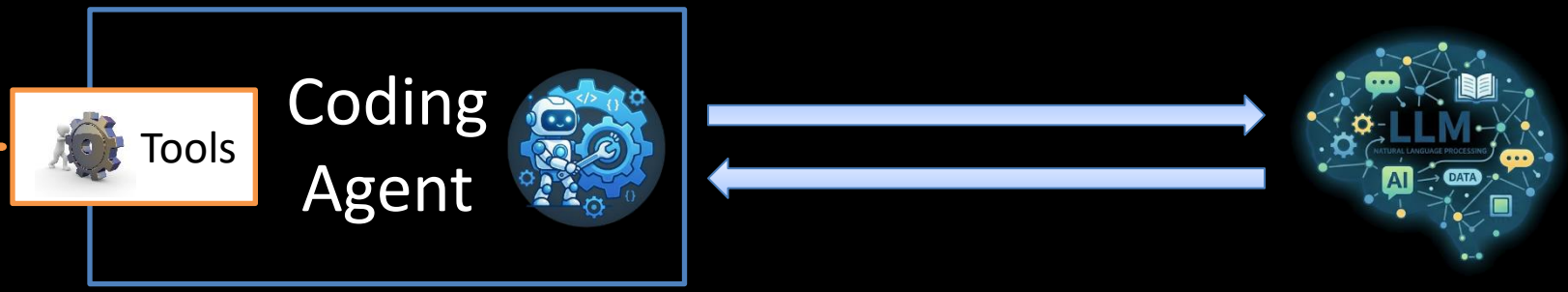


`I can't see the code yet. Please paste the contents of your **Program.cs** file`

How do we solve this?

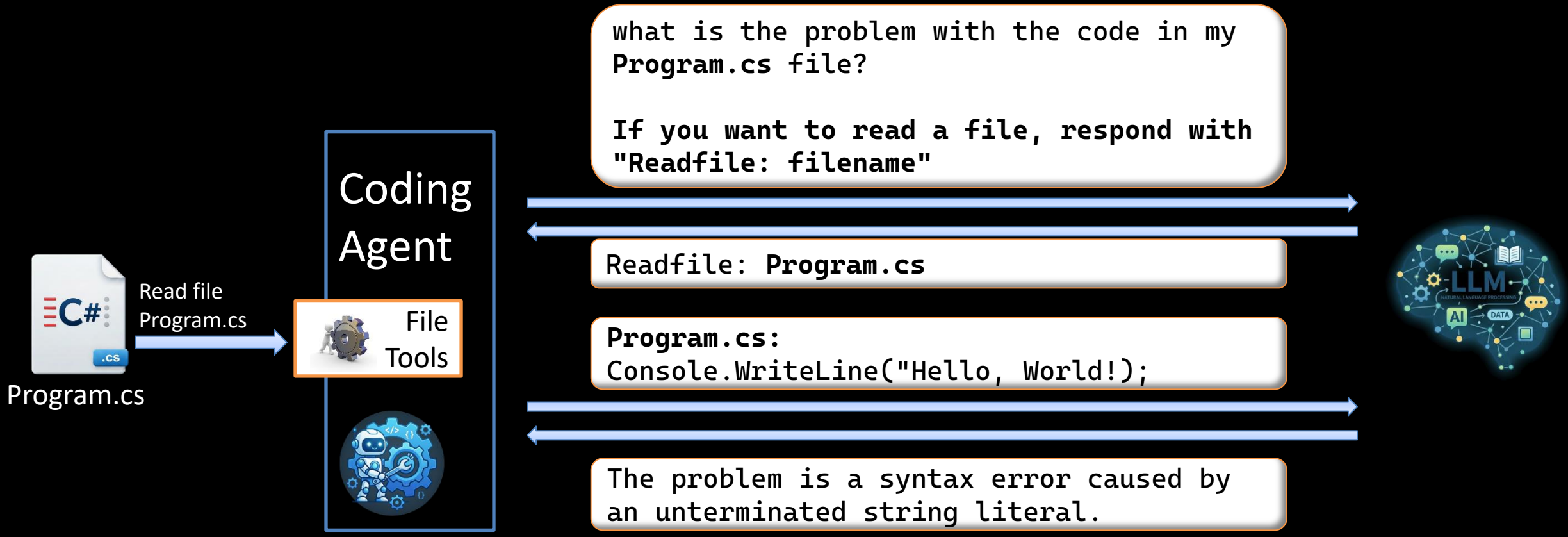
We can give the coding agent **tools**

- File I/O
- Run commands
- Web fetch
- Web search
- …



How does the model use these tools?

The coding agent describes the available tools to the LLM



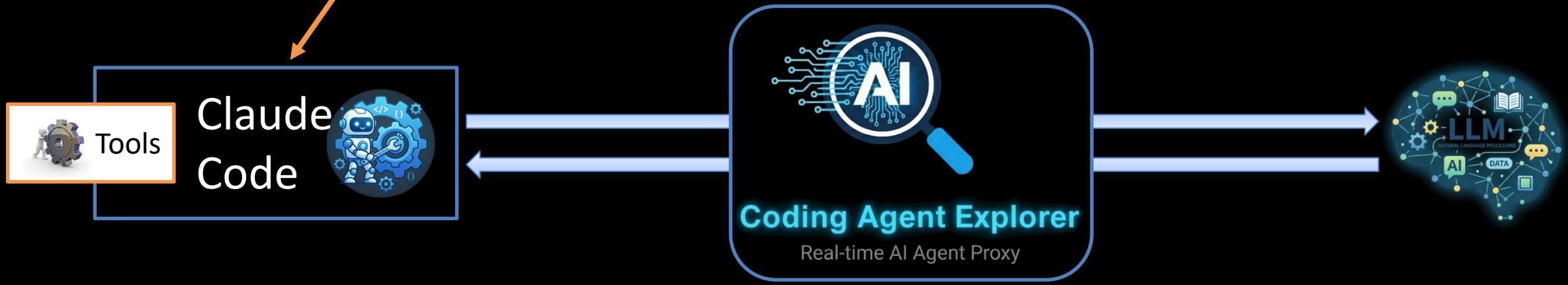
All coding agents work in a similar way!

Demo #2 – Exploring the tools

```
Create a simple FizzBuzz console application in C#
targeting .NET 10.

- Only create Program.cs and the .csproj file.
- Assume Windows environment.
- Do not use bash or execute shell commands.
```

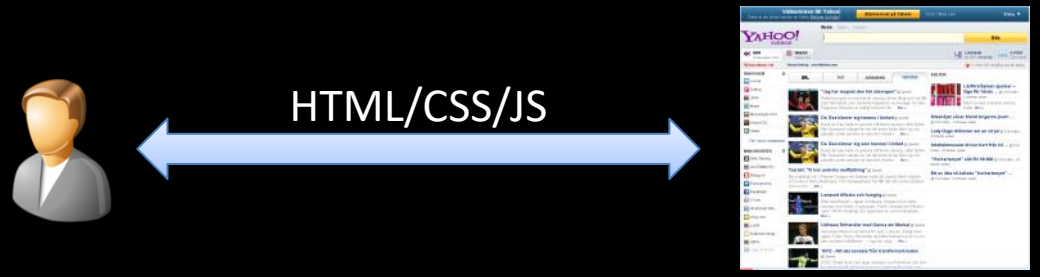




Model Context Protocol (MCP)

MCP

First, we had the web, for humans to consume



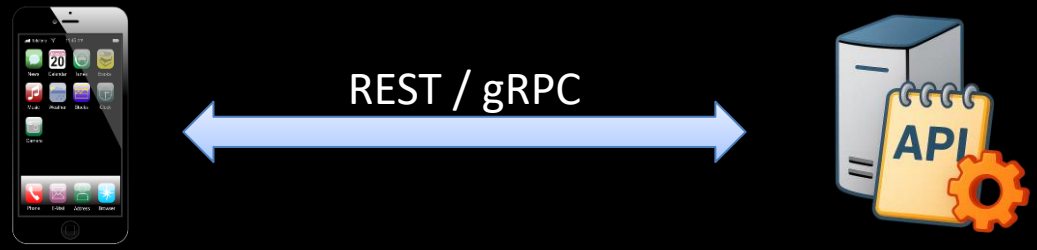
Then came the need for APIs



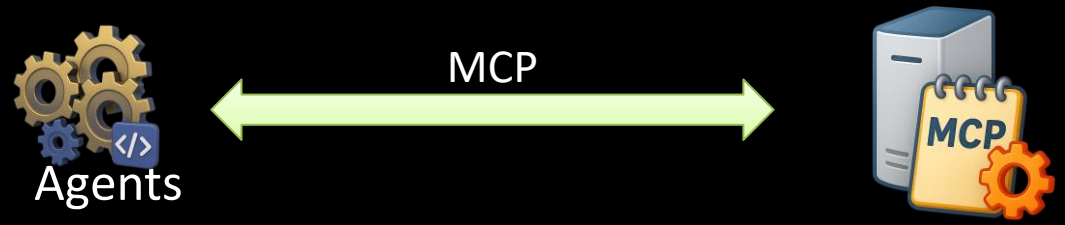
If we already have APIs, why do we need MCP?

MCP

APIs are designed for applications, not agents



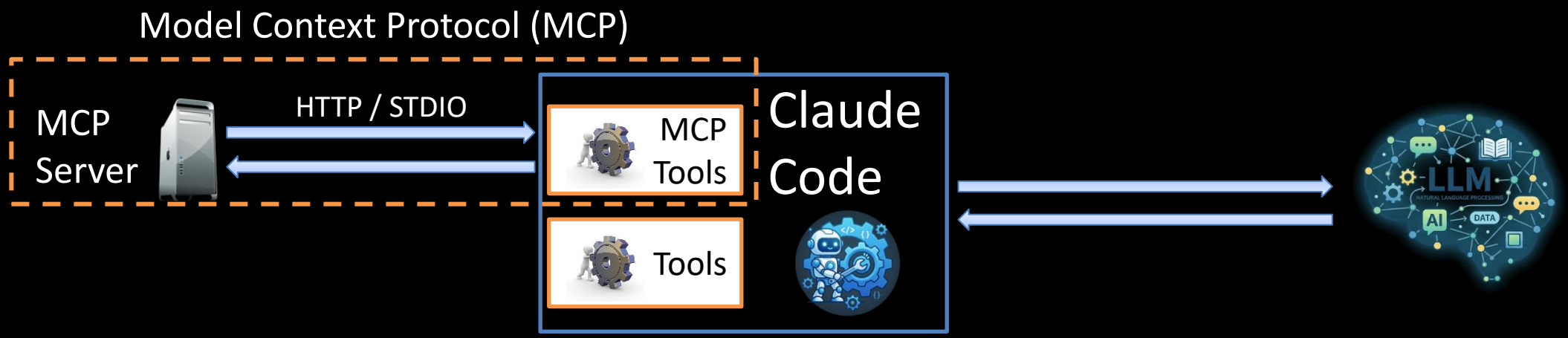
MCP lets you expose services and tools to agents



What does MCP specify?

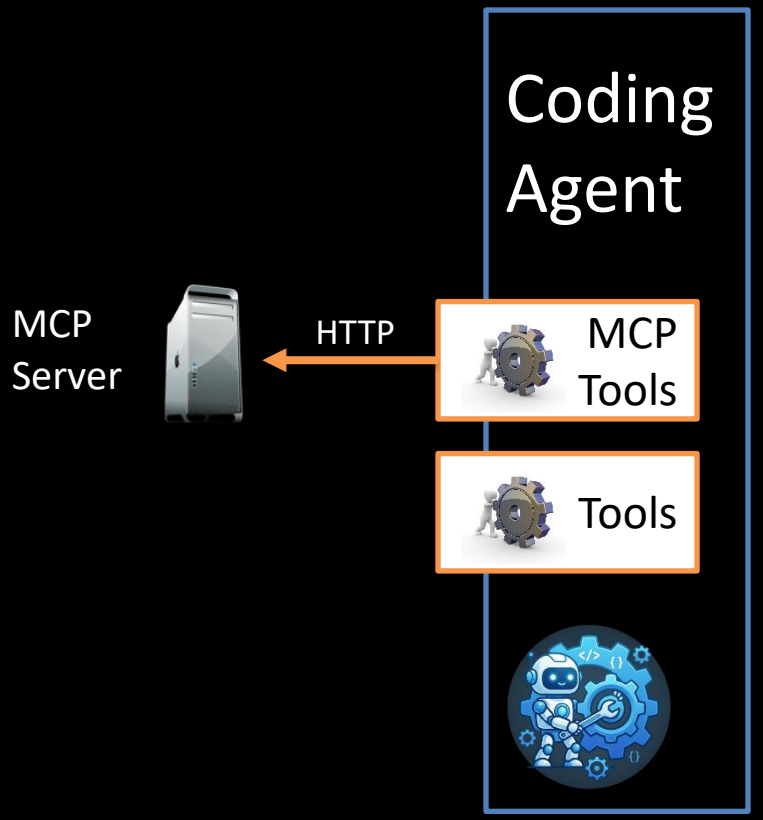
MCP Tools

MCP defines how agents communicate with external tools



How do we use these MCP tools in our coding agent?

MCP tools are exposed to the model as regular tools




```
What is the CloudDebugger?

fetch_CloudDebugger_documentation
Fetch entire documentation file from GitHub
repository: tndata/CloudDebugger.

Useful for general questions. Always call this
tool first if asked about tndata/CloudDebugger.

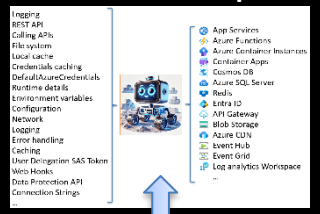
search_CloudDebugger_docs ...
search_CloudDebugger_code ...
fetch_generic_url_content ...
...
```



The model never communicates directly with MCP servers

Demo #3 – Exploring MCP Usage

GitHub Repo



gitmcp.io https://gitmcp.io/tndata/CloudDebugger

What are the main features of the CloudDebugger?

MCP Tools

Claude Code

Coding Agent Explorer
Real-time AI Agent Proxy

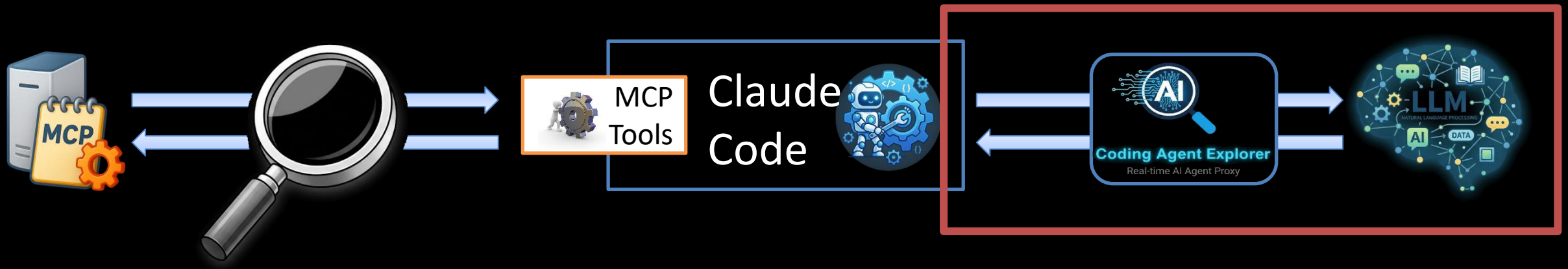
```
claude mcp add --transport http clouddebugger https://gitmcp.io/tndata/CloudDebugger
```

Register MCP server

Exploring the Model Context Protocol

MCP Internals

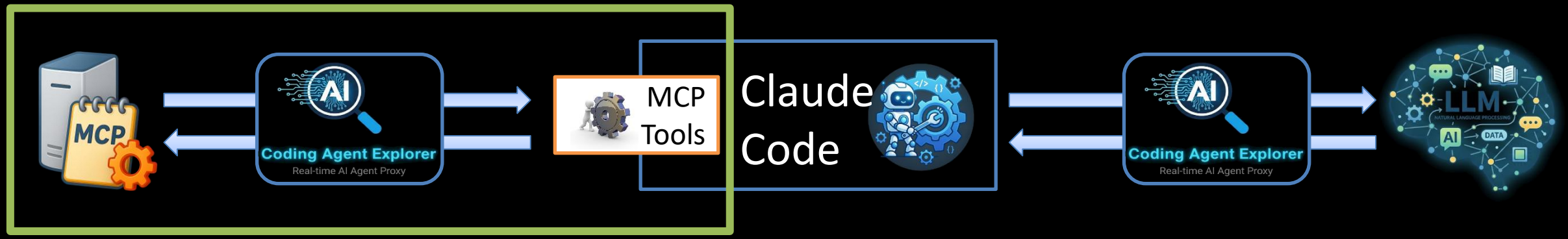
So far, we have only seen how MCP is used by the model



How can we inspect the MCP traffic?

MCP Internals

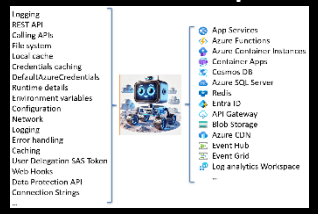
We add the Explorer between the agent and MCP servers



Let's see this in action

Demo #4 – Exploring MCP Traffic

GitHub Repo



gitmcp.io

Destination set in the proxy

What are the main features of the CloudDebugger?

Port: 9999

Coding Agent Explorer
Real-time AI Agent Proxy

MCP Tools

Claude Code

Port: 8888

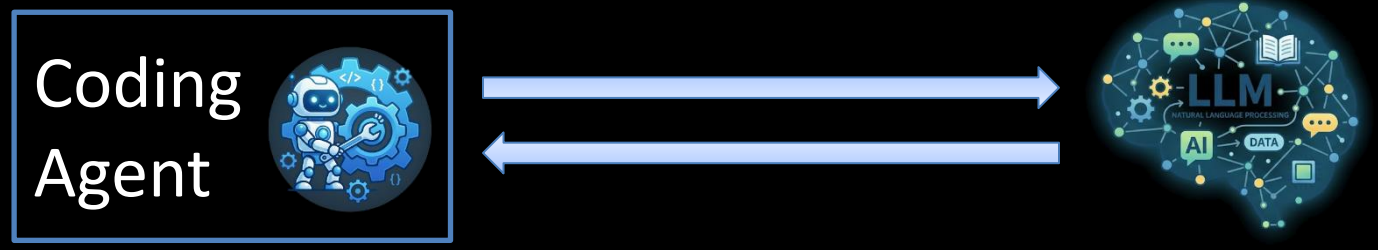
Coding Agent Explorer
Real-time AI Agent Proxy

LLM

```
claude mcp remove clouddebugger
```

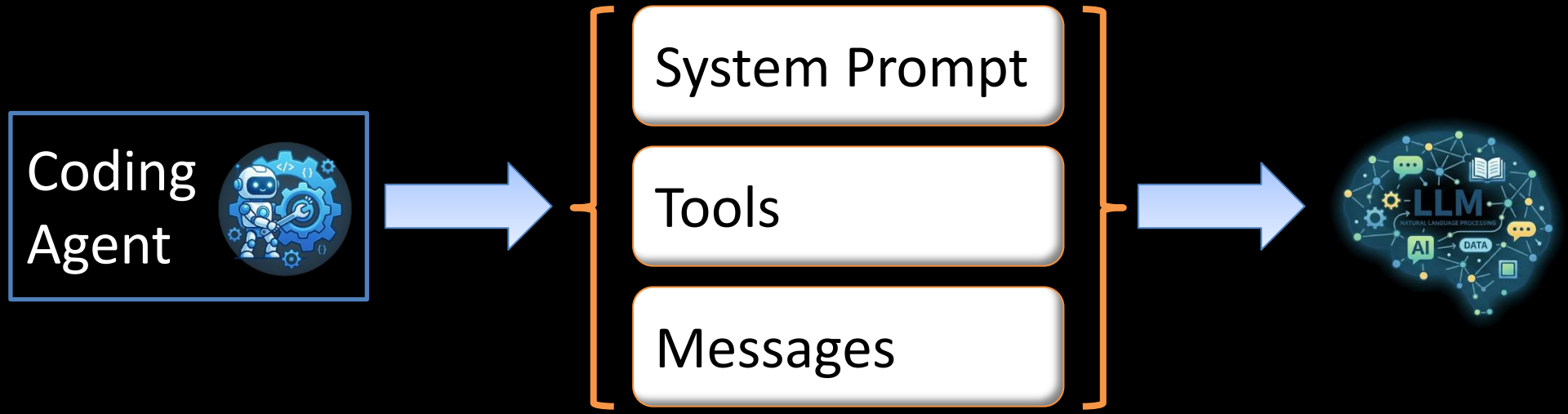
```
claude mcp add --transport http clouddebugger http://localhost:9999
```

The Three Pieces of an LLM Request



The Three Pieces of an LLM Request

Every LLM request consists of three parts:



What are the differences?

System Prompt: The LLM's instruction manual

Coding Agent



Example System Prompt

```
You are an expert C# coding assistant.

Your role is to help developers write, debug, and
improve C# code.

Guidelines:
- Use modern C# features and best practices
- Follow SOLID principles
- Write clean, maintainable code
- Explain your reasoning when making changes
- Always verify your solutions work correctly

Be concise, helpful, and focus on producing working
code.
```

Included in every request



The Three Pieces of an LLM Request

Tools: The list of available tools that the LLM can call

Coding Agent



```
Agent
Artifact
AskUserQuestion
Bash
Edit
Glob
Grep
ListAgents
PowerShell
Read
ReportFindings
ScheduleWakeup
SendFeedback
Skill
ToolSearch
Workflow
Write
...
```

```
Executes a bash command and returns its output.

This tool runs Git Bash (POSIX sh), not cmd.exe or
PowerShell. Use Unix shell syntax: `/dev/null` not
`NUL`, forward slashes, `$VAR` not `%VAR%` or
`$env:VAR`...

- IMPORTANT: Avoid using this tool to run `find`,
`grep`, `cat`, `head`, `tail`, `sed`, `awk`, or
`echo` commands, unless explicitly instructed or
after you have verified that a dedicated tool
cannot accomplish your task. Instead, use the
appropriate dedicated tool as this will provide a
much better experience for the user...
```

Included in every request



Messages: The ongoing conversation

Coding Agent



Included in every request

```
what is the problem with the code in my Program.cs file?

If you want to read a file, respond with "Readfile: filename"
```

```
Readfile: Program.cs
```

```
Program.cs:
Console.WriteLine("Hello, World!);
```

```
The problem is a syntax error caused by an unterminated string literal.
```

```
Thanks! You are a lifesaver!
```

The latest prompt



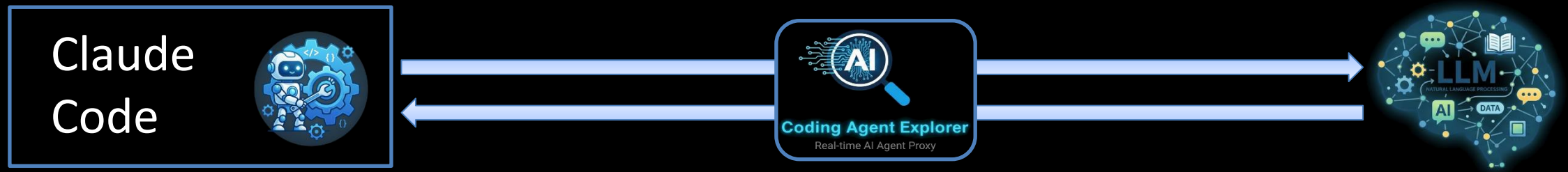
Demo #5 – Exploring The Request

HTTP Inspector

TIME	METHOD	PATH
10:57:13	POST	/v1/messages?beta=true
11:03:23	POST	/v1/messages?beta=true
11:03:28	POST	/v1/messages?beta=true
11:03:30	POST	/v1/messages?beta=true

HTTP Request

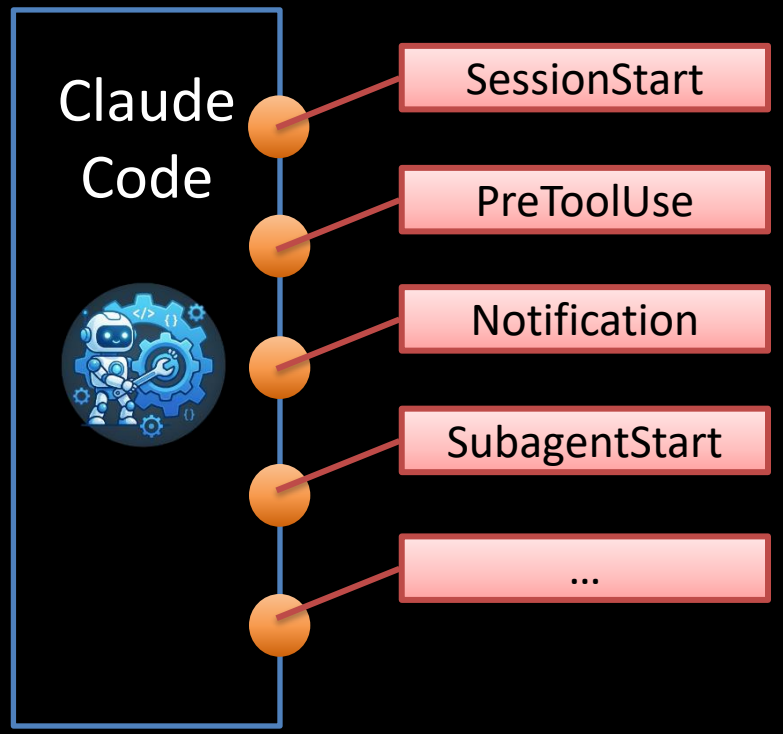

```
▼{
  "model": "claude-sonnet-5-5",
  ▶"messages": [...],
  ▶"system": [...],
  ▶"tools": [...],
  ▶"metadata": {...},
  "max_tokens": 128000,
  ▶"thinking": { adaptive },
  ▶"context_management": {...},
  ▶"safeguards": [...],
  ▶"output_config": {...},
  "stream": true
}
```

Hooks

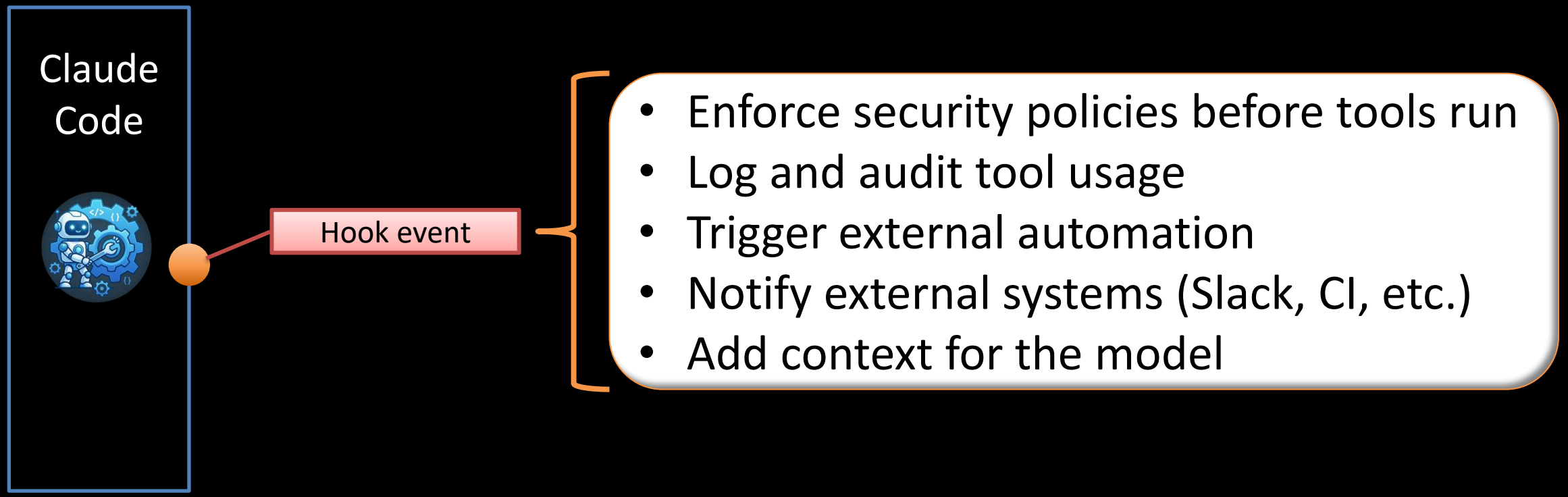
Hooks

Hooks run commands when Claude Code events occur



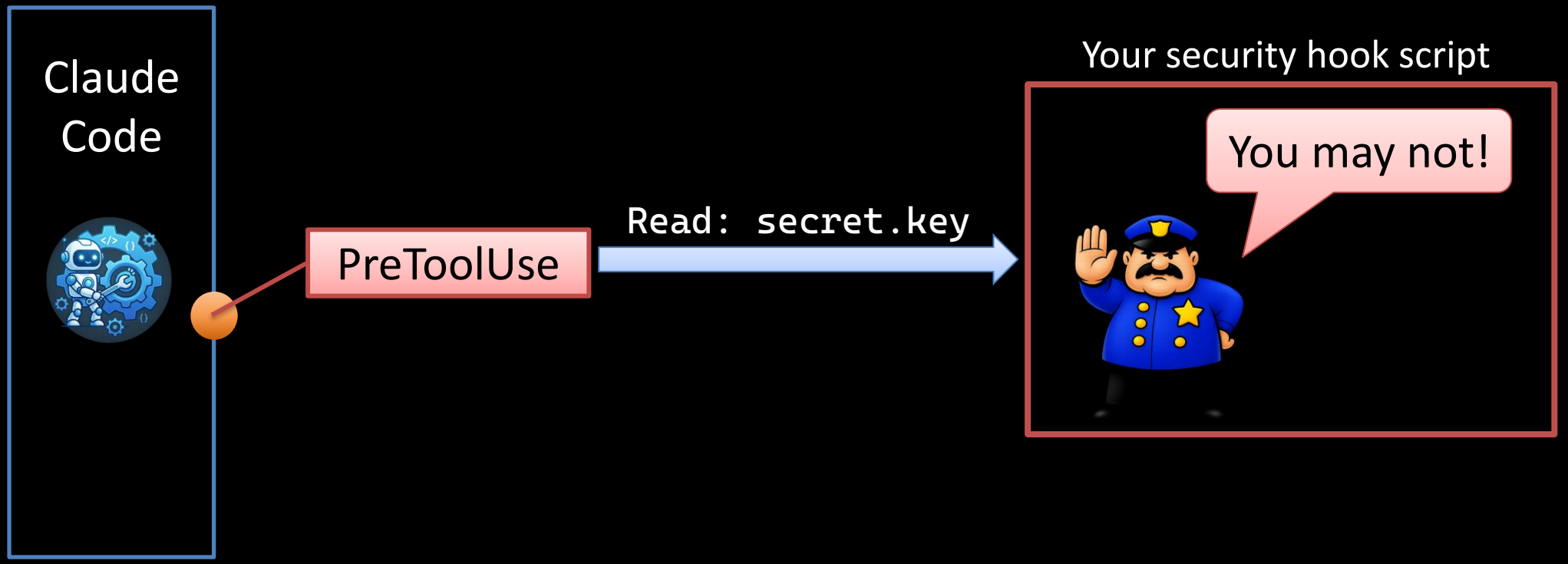
What can hooks be used for?

What you can do with hooks



What does this look like in practice?

Example: Block access to secret files



Which events can we hook into?

Claude Code exposes **30+** hook events

Tool Events
PreToolUse
PostToolUse
PermissionRequest
…

Session Events
SessionStart
SessionEnd
PreCompact
…

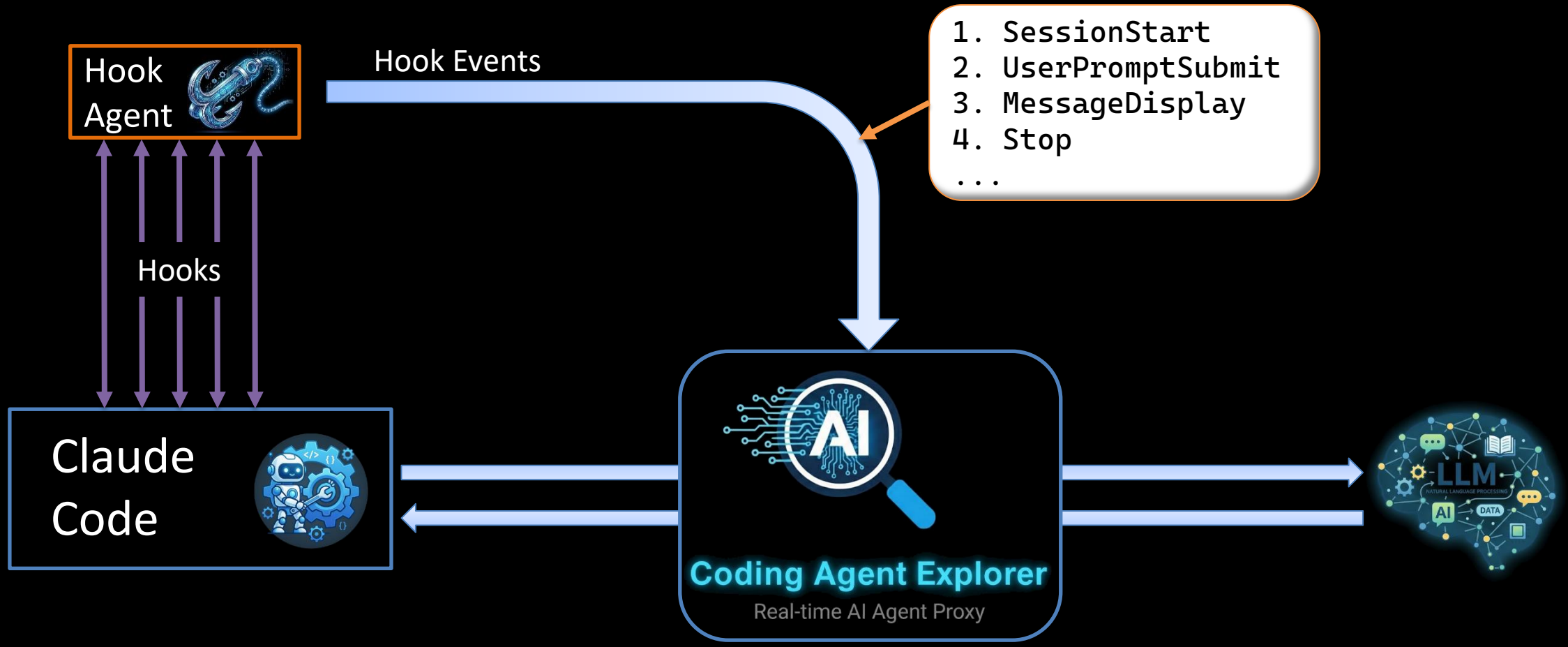


Agent and Task Events
SubagentStart
SubagentStop
TaskCompleted
…

User Interaction
UserPromptSubmit
Notification
MessageDisplay
…

Worktree Events
WorktreeCreate
WorktreeRemove

Demo #6 – Exploring Hooks



Building Your Own Agent in C#

Building Your Own Agent

An agent is just a loop:

```
var agent = new Agent();

while (true)
{
    var input = GetUserInput();

    var response = agent.SendMessage(input);

    Print(response);
}
```

What about the message history (context)?

The context is just a collection

```
var context = new ContextCollection();

while (true)
{
    var input = GetUserInput();

    context.AddUserPrompt(input);

    var response = agent.SendMessage(context);

    context.AddAgentResponse(response);

    Print(response);
}
```

What about tool support?

Adding tool support is easy:

```
while (true)
{
    var input = GetUserInput();
    context.AddUserPrompt(input);

    var response = agent.SendMessage(context);
    context.AddAgentResponse(response);

    while (agent.WantsToUseTool(response))
    {
        // Run the tools, add the results to the context
        agent.ProcessToolCalls(context, response);

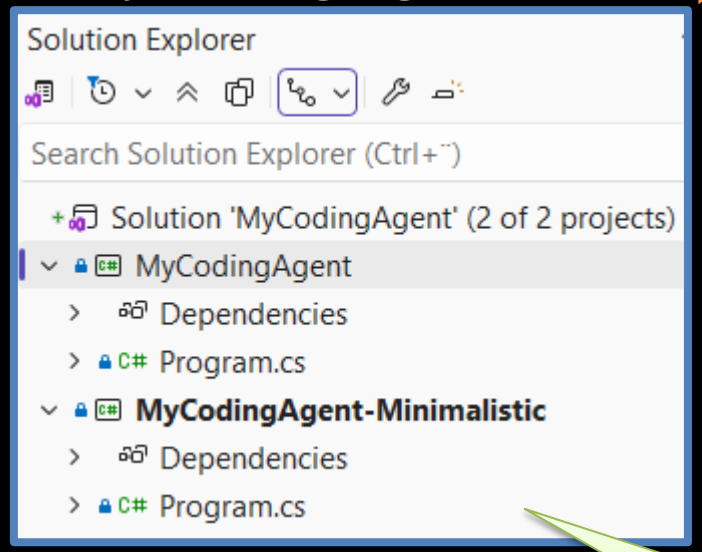
        response = agent.SendMessage(context);
        context.AddAgentResponse(response);
    }
    Print(response);
}
```

Demo #7 – MyCodingAgent.NET



```
Create a simple fizz buzz application in C#, with
program.cs and project file. Do use .NET 10.
```

My Coding Agent .NET



<200 lines of code!

Claude API



Your next steps:

Scan for slides + Resources

- ☑ Book this presentation for my company
- ☑ Run the Coding Agent Explorer
- ☑ Follow Tore on LinkedIn